网络安全技术第二次实验：RSA算法自动分配密钥的聊天室

邹佳衡（2312322）

- 网络安全技术第二次实验：RSA算法自动分配密钥的聊天室
  - 实验环境
  - 实验目的
  - 实验原理
    - 公钥密码体制
    - RSA加解密
  - 代码实现
    - 项目目录结构
    - 运行时生成的文件
    - 服务端启动，生成 RSA 密钥对
    - 客户端连接，获取 RSA 公钥
    - 生成 DES 密钥并通过 RSA 加密传输
    - 聊天室消息通过 DES 加密传输
    - 关于双工通信
  - 实验结果

实验环境

Ubuntu 18.04，VMware WorkStation 17

实验目的

在讨论了传统的对称加密算法DES原理与实现技术的基础上，本章将以典型的非对称密码体系中RSA算法为例，以基于TCP协议的聊天程序加密为任务，系统地进行非对称密码体系RSA算法原理与应用编程技术的讨论和训练。通过练习达到以下的训练目的：

- ① 加深对RSA算法基本工作原理的理解。
- ② 掌握基于RSA算法的保密通信系统的基本设计方法。
- ③ 掌握在Linux操作系统实现RSA算法的基本编程方法。
- ④ 了解Linux操作系统异步IO接口的基本工作原理。

本章编程训练的要求如下。

- ① 要求在Linux操作系统中完成基于RSA算法的自动分配密钥加密聊天程序的编写。
- ② 应用程序保持第三章"基于DES加密的TCP通信"中示例程序的全部功能，并在此基础上进行扩展，实现密钥自动生成，并基于RSA算法进行密钥共享。
- ③ 要求程序实现全双工通信，并且加密过程对用户完全透明。

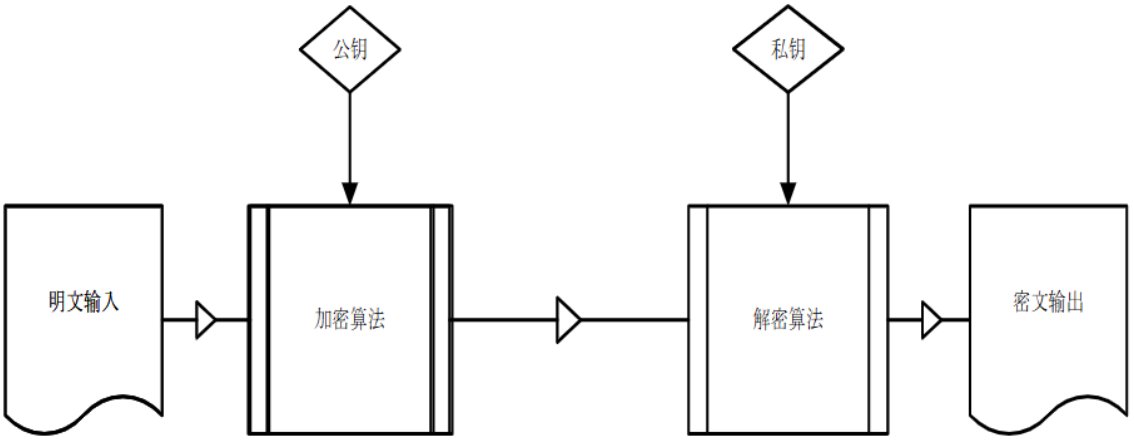
实验原理

公钥密码体制

传统对称密码体制要求通信双方使用相同的密钥，因此应用系统的安全性完全依赖于密钥的保密。针对对称密码体系的缺陷，Differ和Hellman提出了新的密码体系—公钥密码体系，也称为非对称密码体系。在公钥加密系统中，加密和解密使用两把不同的密钥。加密的密钥（公钥）可以向公众公开，但是解密的密钥(私钥)必须是保密的，只有解密方知道。公钥密码体系要求算法要能够保证：任何企图获取私钥的人都无法从公钥中推算出来。公钥密码体制中最著名算法是RSA，以及背包密码、McEliece密码、Diffe_Hellman、Rabin、零知识证明、椭圆曲线、EIGamal算法等。

公钥密码体制如下部分组成：

- (1)明文：作为算法的输入的消息或者数据。
- (2)加密算法：加密算法对明文进行各种代换和变换。
- (3)密文：作为算法的输出，看起来完全随机而杂乱的数据，依赖明文和密钥。对于给定的消息，不同的密钥将产生不同的密文，密文是随机的数据流，并且其意义是无法理解的。
- (4)公钥和私钥：公钥和私钥成对出现，一个用来加密，另一个用来解密。
- (5)解密算法：该算法用来接收密文，解密还原出明文。



RSA加解密

RSA算法是目前最广泛使用的公钥密码算法，其安全性基于大整数质因数分解的困难性——计算两个大质数的乘积很容易，但将一个大合数分解为质因数在计算上不可行。

对于正整数 n，欧拉函数 φ(n) 表示小于 n 且与 n 互质的正整数个数。若 n = p × q，其中 p、q 均为质数，则：

$$\varphi(n) = (p-1)(q-1)$$

若 a 与 n 互质，则：

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$

若 e 与 φ(n) 互质，则存在 d 使得：

$$e \times d \equiv 1 \pmod{\varphi(n)}$$

即 $d = e^{-1} \bmod \varphi(n)$，d 称为 e 的模逆元素。

RSA 密钥对的生成步骤如下：

1. 随机选择两个大质数 p 和 q，本实验中选用 2048 位密钥长度。

2. 计算 n = p × q，n 作为公钥和私钥的公共模数，其位长即为密钥长度。
3. 计算欧拉函数 φ(n) = (p-1)(q-1)。
4. 选择公钥指数 e，满足 1 < e < φ(n) 且 gcd(e, φ(n)) = 1。常用取值为 65537（即 $2^{16}+1$），该值二进制表示中只有两个 1，便于快速加密运算。
5. 计算私钥指数 d，满足 $d \equiv e^{-1} \pmod{\varphi(n)}$，即 d 是 e 关于模 φ(n) 的乘法逆元，通常使用扩展欧几里得算法求解。
6. 公钥 = (n, e)，私钥 = (n, d)。销毁 p、q 和 φ(n)，确保它们不被泄露。

发送方使用接收方的公钥 (n, e) 对明文 M 进行加密：

$$C = M^e \bmod n$$

其中：

- M 为明文，需满足 0 ≤ M < n（超出时需分块加密）
- C 为密文
- e 为公钥指数
- n 为模数

实际应用中，明文 M 在加密前需经过填充处理，以防止选择明文攻击等安全威胁。本实验中采用 OAEP 配合 SHA-1 哈希函数，确保每次加密相同明文也会产生不同的密文。

接收方使用自己的私钥 (n, d) 对密文 C 进行解密：

$$M = C^d \bmod n$$

解密后需去除 OAEP 填充以还原原始明文。

代码实现

项目目录结构

```
chatroom/
├── server/                    # 服务端源码
│   ├── server.cpp             # 服务端主程序
│   ├── RSA.h / RSA.cpp        # RSA 非对称加密模块（共享）
│   ├── DES.h / DES.cpp        # DES 对称加密模块（共享）
│   └── box.h / box.cpp        # DES 算法常量表（共享）
├── client/                    # 客户端源码
    ├── client.cpp             # 客户端主程序
    ├── RSA.h / RSA.cpp        # （同服务端，共享副本）
    ├── DES.h / DES.cpp        # （同服务端，共享副本）
    └── box.h / box.cpp        # （同服务端，共享副本）
```

分别用以下两条命令编译服务端和客户端：

```
g++ -std=c++11 -pthread -o server server.cpp DES.cpp RSA.cpp box.cpp -lcryptopp

g++ -std=c++11 -pthread -o client client.cpp DES.cpp RSA.cpp box.cpp -lcryptopp
```

运行时生成的文件

文件	位置	说明
public.key	服务端	RSA 公钥文件（运行开始时生成，通过 Socket 传给客户端）
private.key	服务端	RSA 私钥文件（运行开始时生成，仅保存在服务端，绝不外传）
client_des.key	客户端	客户端生成的 DES 会话密钥的本地备份（8 字节随机字符串）

服务端启动，生成 RSA 密钥对

服务端首先调用 Crypto++ 库生成 2048 位 RSA 密钥对，分别保存为 public.key 和 private.key 文件，并将公钥文件 Base64 编码后缓存到内存中。

server/main() 中 RSA 密钥初始化：

```
int main() {
    // RSA key initialization
    string pubFile = "public.key";
    string privFile = "private.key";

    cout << "Generating RSA keys (this may take a few seconds)..." << endl;
    generateRSAKeys(2048, pubFile, privFile);          // 生成 2048 位密钥对

    cachedPubKeyStr = getKeyAsString(pubFile);          // 公钥 Base64 编码为字符串
    cout << "RSA keys generated. Public key length: "
         << cachedPubKeyStr.length() << " bytes" << endl;

    // 创建 Socket 监听 8080 端口...
    int server_socket = socket(AF_INET, SOCK_STREAM, 0);
    ...
}
```

RSA.cpp 中生成密钥对的函数：

```
void generateRSAKeys(unsigned int keyLength,
                     const string& pubFile, const string& privFile) {
    AutoSeededRandomPool rng;
    InvertibleRSAFunction params;
    params.GenerateRandomWithKeySize(rng, keyLength);   // 随机生成 p、q 质数

    RSA::PrivateKey privateKey(params);                 // 从参数构造私钥 (n,d)
    RSA::PublicKey publicKey(params);                   // 从参数构造公钥 (n,e)
```

```
    FileSink privSink(privFile.c_str());
    privateKey.Save(privSink);                          // 私钥保存到文件

    FileSink pubSink(pubFile.c_str());
    publicKey.Save(pubSink);                            // 公钥保存到文件
}
```

getKeyAsString() 将密钥文件编码为可传输的字符串：

```
string getKeyAsString(const string& keyFile) {
    string binaryKey, base64Key;
    FileSource fs(keyFile.c_str(), true, new StringSink(binaryKey));    // 读取二进
制
    StringSource ss(binaryKey, true,
                    new Base64Encoder(new StringSink(base64Key)));      // Base64
编码
    return base64Key;
}
```

客户端连接，获取 RSA 公钥

当客户端 TCP 连接建立后，服务端通过 acceptNewClient() 将公钥发送给客户端。传输协议采用自定义的标记格式：[PUBKEY_BEGIN] + 长度 + [PUBKEY_END] + 公钥数据。

server/acceptNewClient() 中发送公钥：

```
void acceptNewClient(int server_socket) {
    // ... 接受 TCP 连接，记录客户端 IP ...

    // 发送欢迎消息（明文，此时尚未交换 DES 密钥）
    string tips = "=== Welcome to the chat room! ===\r\n";
    sendToClient(client_socket, tips);

    // 发送公钥：先发送包头 [PUBKEY_BEGIN]长度[PUBKEY_END]，再发送公钥数据本体
    string header = "[PUBKEY_BEGIN]" + to_string(cachedPubKeyStr.length())
                  + "[PUBKEY_END]";
    send(client_socket, header.c_str(), header.length(), 0);
    send(client_socket, cachedPubKeyStr.c_str(), cachedPubKeyStr.length(), 0);
}
```

client/receivePublicKey() 中接收并解析公钥：

```
bool receivePublicKey() {
    string full_received_data;
    // ...
    // 1. 在接收数据中查找 [PUBKEY_BEGIN] 和 [PUBKEY_END] 标记
```

```
    size_t start_pos = full_received_data.find("[PUBKEY_BEGIN]");
    size_t end_pos   = full_received_data.find("[PUBKEY_END]");

    // 2. 提取两个标记之间的数字作为公钥长度
    size_t len_start = start_pos + strlen("[PUBKEY_BEGIN]");
    string len_str = full_received_data.substr(len_start, end_pos - len_start);
    expected_pubkey_len = stoi(len_str);

    // 3. 取 [PUBKEY_END] 之后的数据作为公钥本体
    size_t pubkey_start = end_pos + strlen("[PUBKEY_END]");
    pubkey_str = full_received_data.substr(pubkey_start);

    // 4. 收集足够长度后，保存为 public.key 文件
    if ((int)pubkey_str.length() >= expected_pubkey_len) {
        pubkey_str = pubkey_str.substr(0, expected_pubkey_len);
        saveKeyFromString(pubkey_str, "public.key");    // Base64 解码后写文件
    }
}
```

saveKeyFromString() 将 Base64 字符串还原为密钥文件：

```
void saveKeyFromString(const string& keyStr, const string& keyFile) {
    string decodedKey;
    StringSource ss(keyStr, true,
                    new Base64Decoder(new StringSink(decodedKey)));  // Base64 解
码
    FileSink fs(keyFile.c_str());
    fs.Put((const byte*)decodedKey.data(), decodedKey.size());      // 写入文件
}
```

生成 DES 密钥并通过 RSA 加密传输

DES 密钥交换是本实验混合加密方案的核心环节。客户端生成 8 字节随机 DES 密钥，用 RSA 公钥加密后发送，服务端用 RSA 私钥解密获得 DES 密钥。整个过程保证了 DES 密钥不会以明文出现在网络上。

client/sendDESKey() —— 客户端用 RSA 公钥加密 DES 密钥并发送：

```
bool sendDESKey(const string& pubKeyFile) {
    // RSA-OAEP 加密 8 字节 DES 密钥，返回 Base64 编码的密文
    string encryptedKey = rsaEncrypt(desKey, pubKeyFile);

    // 封装为标记格式发送
    string keyMsg = "[DESKEY_BEGIN]" + encryptedKey + "[DESKEY_END]\r\n";
    send(client_socket, keyMsg.c_str(), keyMsg.length(), 0);
}
```

RSA.cpp/rsaEncrypt() —— RSA-OAEP 公钥加密：

```
string rsaEncrypt(const string& plaintext, const string& pubFile) {
    AutoSeededRandomPool rng;
    RSA::PublicKey publicKey;
    FileSource fs(pubFile.c_str(), true);
    publicKey.Load(fs);                                  // 加载公钥

    string ciphertext;
    RSAES_OAEP_SHA_Encryptor encryptor(publicKey);       // OAEP 填充 + SHA-1
    StringSource ss(plaintext, true,
        new PK_EncryptorFilter(rng, encryptor,
            new StringSink(ciphertext)));                // RSA 加密

    string encoded;
    StringSource ss_encode(ciphertext, true,
        new Base64Encoder(new StringSink(encoded)));     // Base64 编码
    return encoded;
}
```

server/processMessage() 识别 DES 密钥标记 → handleDESKey() 用 RSA 私钥解密:

```
// processMessage() 中识别 DES 密钥消息
void processMessage(int client_socket, const string& data) {
    size_t key_start = data.find("[DESKEY_BEGIN]");
    size_t key_end   = data.find("[DESKEY_END]");

    if (key_start != string::npos && key_end != string::npos) {
        // 提取两个标记之间的 RSA 密文
        size_t encrypted_start = key_start + strlen("[DESKEY_BEGIN]");
        string encryptedKey = data.substr(encrypted_start,
                                          key_end - encrypted_start);
        handleDESKey(client_socket, encryptedKey, "private.key");
        return;
    }
    // ... 否则按聊天消息处理 ...
}

// handleDESKey() 用 RSA 私钥解密获得 DES 密钥
bool handleDESKey(int client_socket, const string& encryptedKey,
                  const string& privKeyFile) {
    string decryptedKey = rsaDecrypt(encryptedKey, privKeyFile); // RSA 私钥解密

    if (decryptedKey.length() != 8) {                    // 验证 DES 密钥长度
        cout << "Invalid DES key length: " << decryptedKey.length() << endl;
        return false;
    }

    clientDESkey[client_socket] = decryptedKey;          // 存储该客户端的 DES 密钥
    awaitingDESKey[client_socket] = false;

    sendToClient(client_socket, "[DESKEY_OK]");          // 加密确认
```

```
    return true;
}
```

RSA.cpp/rsaDecrypt() —— RSA-OAEP 私钥解密：

```
string rsaDecrypt(const string& encoded_ciphertext, const string& privFile) {
    AutoSeededRandomPool rng;
    RSA::PrivateKey privateKey;
    FileSource fs(privFile.c_str(), true);
    privateKey.Load(fs);                                // 加载私钥

    string ciphertext;
    StringSource ss_decode(encoded_ciphertext, true,
        new Base64Decoder(new StringSink(ciphertext))); // Base64 解码

    string decryptedtext;
    RSAES_OAEP_SHA_Decryptor decryptor(privateKey);
    StringSource ss(ciphertext, true,
        new PK_DecryptorFilter(rng, decryptor,
            new StringSink(decryptedtext)));            // RSA 解密 + 去 OAEP 填充
    return decryptedtext;
}
```

聊天室消息通过 DES 加密传输

DES 密钥交换完成后，所有聊天消息均采用 **DES 加密 → Base64 编码 → 追加 \r\n 分隔符** 的格式传输。Base64 编码保证了二进制密文不会与 \r\n 分隔符发生冲突。

DES.cpp/desEncryptString() —— DES 字符串加密（PKCS#7 填充 + ECB 模式）：

```
string desEncryptString(const string& plaintext, const string& key) {
    // 1. 密钥标准化为 8 字节
    string paddedKey = key;
    if (paddedKey.length() < 8)
        paddedKey.append(8 - paddedKey.length(), '\0'); // 不足补 0
    else if (paddedKey.length() > 8)
        paddedKey = paddedKey.substr(0, 8);             // 超出截断
    bitset<64> keyBits = stringToBitset(paddedKey);

    // 2. PKCS#7 填充：使明文长度为 8 的倍数
    string paddedPlaintext = pkcs7Pad(plaintext);

    // 3. 分块加密（ECB 模式）
    string ciphertext;
    for (size_t i = 0; i < paddedPlaintext.length(); i += 8) {
        string block = paddedPlaintext.substr(i, 8);
        bitset<64> plaintextBits = stringToBitset(block);
        bitset<64> ciphertextBits = desEncryptBlock(plaintextBits, keyBits);
        ciphertext += bitsetToString(ciphertextBits);
```

```
    }
    return ciphertext;
}
```

client/main() 发送消息 —— DES 加密 → Base64 编码 → \r\n：

```
// 发送昵称
string nickname_msg = client_name + "\r\n";
string encryptedNickname = desEncryptString(nickname_msg, desKey);
string encodedNickname = base64Encode(encryptedNickname) + "\r\n";
send(client_socket, encodedNickname.c_str(), encodedNickname.length(), 0);

// 主循环：发送聊天消息
while (connected) {
    getline(cin, message);
    // ...
    message += "\r\n";
    string encryptedMsg = desEncryptString(message, desKey);  // DES 加密
    string encodedMsg = base64Encode(encryptedMsg) + "\r\n";  // Base64 编码 + 分
隔符
    send(client_socket, encodedMsg.c_str(), encodedMsg.length(), 0);
}
```

server/processMessage() 接收消息 —— \r\n 分割 → Base64 解码 → DES 解密：

```
void processMessage(int client_socket, const string& data) {
    // ... 先检查 [DESKEY_BEGIN] 标记（见上一节）...

    // Base64 解码 → DES 解密
    string rawCiphertext = base64Decode(data);
    string decryptedData = desDecryptString(rawCiphertext,
                                            clientDESkey[client_socket]);

    // 去除末尾 \r\n
    while (!decryptedData.empty() &&
           (decryptedData.back() == '\r' || decryptedData.back() == '\n'))
        decryptedData.pop_back();

    // 判断是昵称还是普通聊天消息
    if (clientNames.find(client_socket) == clientNames.end()) {
        clientNames[client_socket] = decryptedData;     // 注册昵称
        // 广播加入通知...
    } else {
        // 格式化时间戳，广播聊天消息
        string formatted_msg = "<" + time_str + " "
                             + clientNames[client_socket] + "> "
                             + decryptedData + "\r\n";
        broadcastMessage(formatted_msg, client_socket);
```

```
    }
}
```

server/broadcastMessage() 服务端广播 —— 用每个客户端的各自 DES 密钥加密：

```
void broadcastMessage(const string& message, int sender) {
    lock_guard<mutex> lock(g_clients_mutex);
    // ...
    for (int client : g_clients) {
        if (client != sender) {
            auto it = clientDESkey.find(client);
            if (it != clientDESkey.end() && !it->second.empty()) {
                // 用该客户端的 DES 密钥加密
                string encryptedMsg = desEncryptString(formatted_msg, it->second);
                string encodedMsg = base64Encode(encryptedMsg) + "\r\n";
                send(client, encodedMsg.c_str(), encodedMsg.length(), 0);
            }
        }
    }
}
```

server/handleClientMessages() TCP 接收缓冲：

```
void handleClientMessages() {
    while (true) {
        // ... select() 多路复用监听所有客户端 ...
        for (int client_socket : clients_copy) {
            if (FD_ISSET(client_socket, &readfds)) {
                char buffer[2048];
                int bytes_received = recv(client_socket, buffer, ...);

                // 关键：累积到该客户端的独立接收缓冲区
                clientRecvBuf[client_socket] += string(buffer, bytes_received);

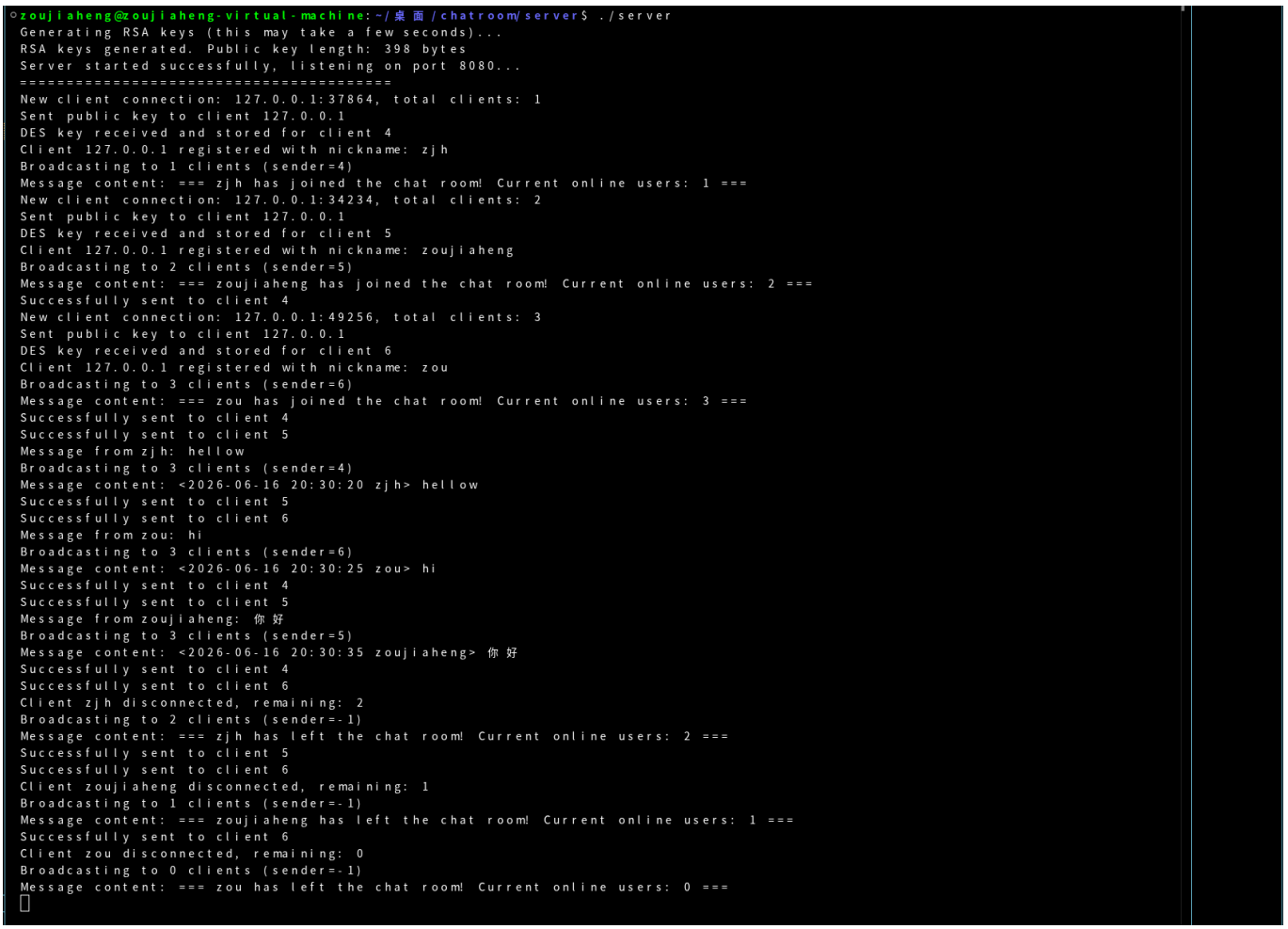
                // 按 \r\n 分割出完整消息，逐条处理
                string& buf = clientRecvBuf[client_socket];
                size_t pos;
                while ((pos = buf.find("\r\n")) != string::npos) {
                    string message = buf.substr(0, pos);
                    buf = buf.substr(pos + 2);          // 移除已处理消息
                    if (!message.empty())
                        processMessage(client_socket, message);
                }
            }
        }
    }
}
```

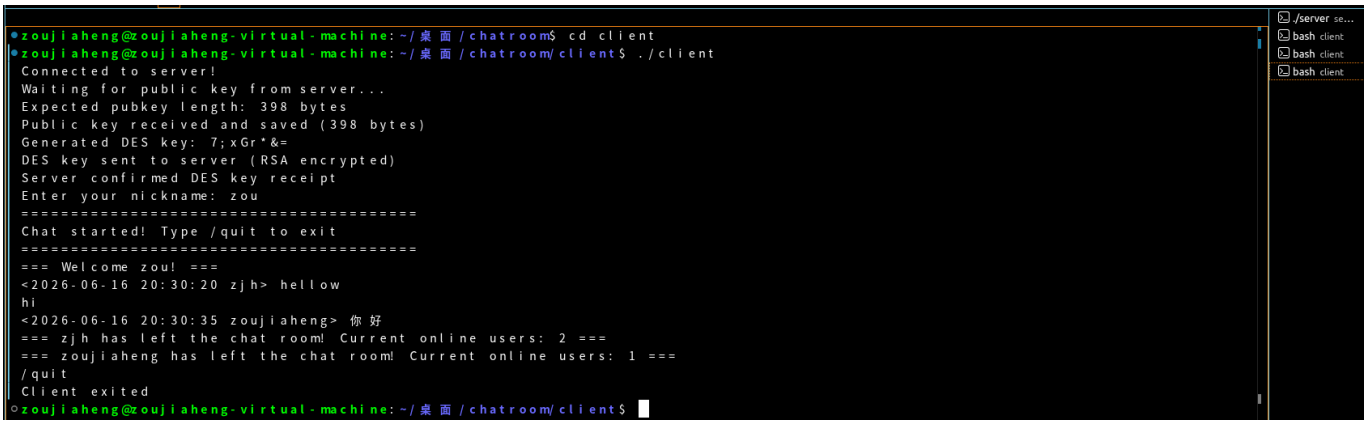
关于双工通信

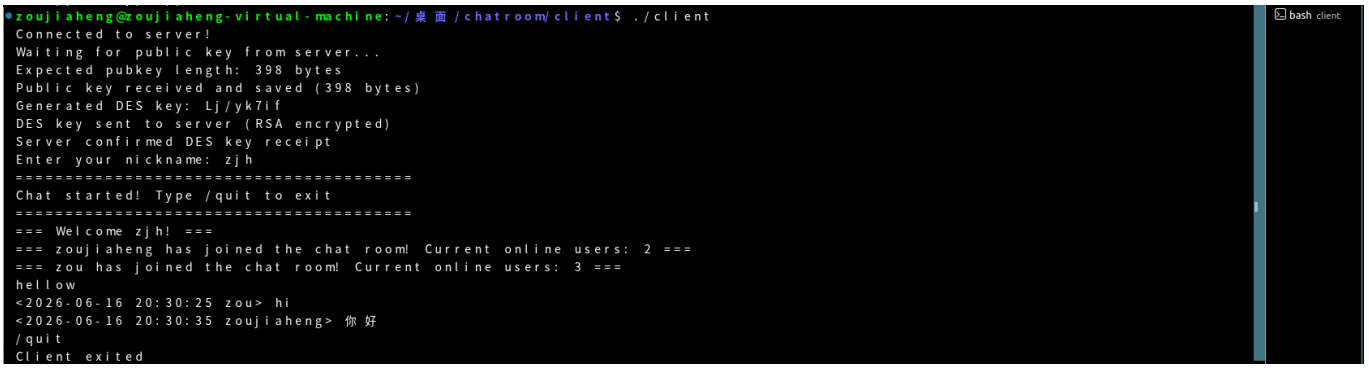
我们的实验代码实现了双工通信，客户端一个线程负责接收，主线程负责发送，两个方向同时进行、互不阻塞。服务端一个线程用 select() 多路复用监听所有客户端的可读事件，主线程循环 accept() 新连接，收发可以同时发生。

实验结果

具体的实验结果可以看录屏，这里只展示截图：



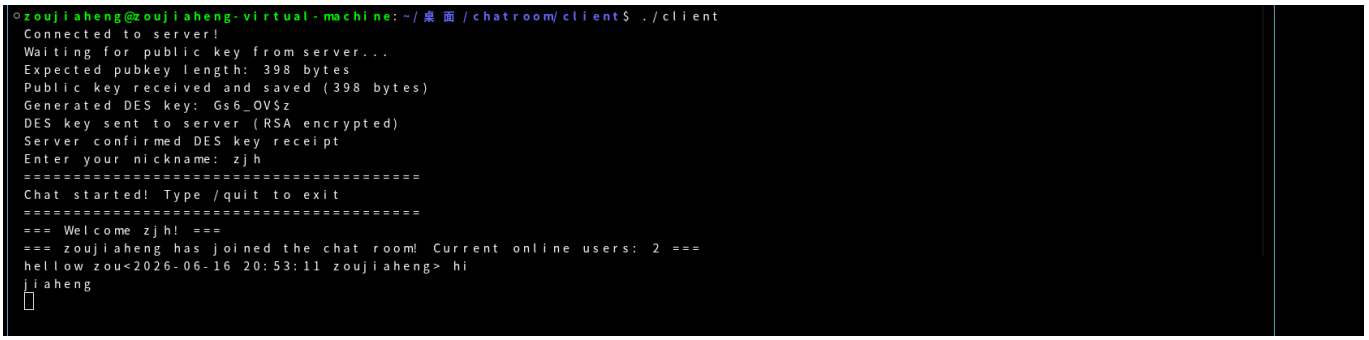






如图，几个客户端可以正常进行加密通讯。

双工通信：





如图，我们在一方打字时让另一方发送信息，虽然打字突然被消息打断，但是消息仍然可以不受影响正常输入和发布。